PitLane (base Swift)

Integrantes

Juan Sebastián Arias Rueda

Sofía Bermúdez González

Sofía Rudas Sáenz

Juan José Trujillo Rendón

Profesor

John Harold Morales Vélez

Materia

Procesadores de Lenguajes

Repositorio

https://github.com/Yogwito/PitLane.git

Universidad Autónoma de Manizales

Manizales, Caldas

2026

Contenido

1. Introducción 1
2. Análisis del lenguaje base: Swift 1
   2.1 Componentes léxicos 1
   2.2 Estructuras de control 2
   2.3 Tabla léxica de Swift 2
3. Diseño de PitLane 3
   3.1 Decisiones generales 3
   3.2 Equivalencias PitLane = Swift 4
   3.3 Funciones predefinidas 6
   3.4 Reglas del lenguaje 6
   3.5 Tabla léxica de PitLane 7
   3.6 Programas de ejemplo 7
4. Gramática de PitLane (EBNF) 9
   4.1 Convenciones 9
   4.2 Reglas 9
   4.3 Precedencia y asociatividad 11
   4.4 Restricciones de contexto 12
   4.5 Notas para el analizador sintáctico 12
5. Errores y límites del procesador 12
   5.1 Formato de los errores 12
   5.2 Límites de ejecución 13
6. Alcance y estado 13
7. Referencias 14

1. Introducción

PitLane es un lenguaje de programación pequeño elaborado en base a vocabulario de Fórmula 1. Fue diseñado a partir del nivel léxico y sintáctico del lenguaje Swift con el uso de palabras reservadas, operadores y reglas gramaticales propios.

El procesador de PitLane está escrito en Swift y funciona como un intérprete realizando el siguiente proceso: lee un archivo de texto, realiza el análisis léxico y sintáctico y finalmente ejecuta el programa mostrando su salida. Los errores léxicos, sintácticos y de ejecución se informan con el número de la línea donde ocurren.

2. Análisis del lenguaje base: Swift

2.1 Componentes léxicos

Swift delimita los bloques con llaves, no con la indentación; el punto y coma es opcional al final de una sentencia y el salto de línea la termina.

A diferencia de otros lenguajes **print()** y **readLine()** son funciones de la librería estándar y no palabras reservadas.

Componentes léxicos de Swift

Categoría	Elementos
Declaración	let, var, func, class, init, self
Condicionales	if, else, switch, case, default, guard
Bucles	while, repeat, for, in
Control de flujo	break, continue, return
Excepciones	do, try, catch, throw, throws, defer
Literales	true, false, nil
Operadores aritméticos	+ - * / %
Asignación	= += -= *= /= %=
Relacionales	== != < > <= >=
Lógicos	&& || !
Rangos	..< y ...
Otros operadores	-> (tipo de retorno), ? (opcional o ternario)
Delimitadores	( ) { } [ ] , ; : . y el salto de línea
Tipos básicos	Int, Double, String, Bool, arreglos como [Int]
Comentarios	// de línea y /* ... */ de bloque

2.2 Estructuras de control

Swift no tiene un **finally**: la limpieza se hace con **defer**, que se ejecuta al salir del bloque. Además, **throw** solo acepta valores que cumplan el protocolo **Error**, PitLane simplifica esta regla y permite lanzar cualquier valor.

Estructuras de control de Swift

Estructura	Forma en Swift
Condicional	if cond { ... } else if cond { ... } else { ... }
Bucle condicional	while cond { ... }
Bucle con repetición	repeat { ... } while cond
Bucle con contador	for i in a...b { ... }
Interrupción de bucles	break y continue
Función	func nombre(a: Int) -> Int { return a }
Clase	class C { var x: Int; init(x: Int) { self.x = x } }
Lanzar un error	throw error
Capturar errores	do { try f() } catch { ... }
Limpieza al salir	defer { ... }

2.3 Tabla léxica de Swift

Token	Patrón (regex)	Ejemplo
ID	`[a-zA-Z_][a-zA-Z0-9_]*`	counter
INT_NUMBER	`[0-9]+`	20
REAL_NUMBER	`[0-9]+\.[0-9]+`	3.14
STRING	`"([^"\\]|\\.)*"`	"Hello"
BOOL	`true|false`	true
KEYWORD	`let|var|func|class|if|else|while|for|in|...`	func
ARITH_OP	`[+\-*/%]`	+
ASSIGN	`=|\+=|-=|\*=|/=|%=`	=
REL_OP	`==|!=|<=|>=|<|>`	>=
LOGIC_OP	`&&|||!`	&&
RANGE	`\.\.<|\.\.\.`	..<
ARROW	`->`	->
LPAREN / RPAREN	`\( / \)`	(
LBRACE / RBRACE	`\{ / \}`	{
LBRACKET / RBRACKET	`\[ / \]`	[

Token	Patrón (regex)	Ejemplo
COMMA, COLON, DOT, SEMI	`, : \. ;`	,
COMMENT	`//[^\n]*`	// nota
COMMENT_BLOCK	`/\*([^*]|\*+[^*/])*\*+/`	/* nota */
NEWLINE	`\n`	(salto de línea)
WHITESPACE	`[ \t\r]+`	(se ignora)

Reglas que sigue el analizador léxico de Swift y que PitLane conserva:

- Se aplica la coincidencia más larga primero: **<=** antes que **<** y **==** antes que **=**
- **REAL_NUMBER** se prueba antes que **INT_NUMBER**
- Una palabra es **KEYWORD** si está en la lista de reservadas y **ID** si no
- En Swift los comentarios de bloque pueden anidarse, por lo que la expresión regular de la tabla solo los aproxima. PitLane no tiene comentarios de bloque

3. Diseño de PitLane

3.1 Decisiones generales

El vocabulario completo de PitLane esta basado en su totalidad en la Fórmula 1, ninguna palabra reservada coincide con las palabras obvias, por ejemplo, el cierre de bloque no es **fin** sino **bandera_cuadros**, la bandera con la que termina una carrera.

Tabla 4. Decisiones de diseño

Decisión	Valor
Lenguaje base	Swift
Nombre del lenguaje	PitLane
Tema del vocabulario	Fórmula 1
Lenguaje del procesador	Swift (analizador léxico, analizador sintáctico e intérprete)
Versión mínima de Swift	5.9; funciona en macOS, Linux y Windows
Cierre de bloques	bandera_cuadros, siempre
Notación de la gramática	EBNF
Extensión de los programas	.txt
Estilo de ejecución	Se interpreta, no se traduce a otro lenguaje
Línea de comandos	PitLane [--solo-salida] <archivo.txt>

El flujo de ejecución tiene tres fases:

- **Análisis léxico:** lee el **.txt**, produce los tokens con su tipo y su línea.

- **Análisis sintáctico:** valida la gramática y construye el árbol sintáctico (AST).
- **Ejecución:** el intérprete recorre el árbol y muestra la salida del programa.

Por defecto el procesador imprime las tres fases con sus encabezados **(=== Tokens ===, === Sintaxis === y === Salida ===);** con la opción **--solo-salida** imprime únicamente la salida del programa.

Un error en cualquier fase detiene el proceso con código de salida 1

3.2 Equivalencias PitLane = Swift

A continuación, se relaciona cada palabra de PitLane con su equivalente en swift

Los símbolos **+ - * / % ( ) [ ] , : .** no cambian.

PitLane	Swift	Construcción
parrilla	var	declarar variable
pole	let	declarar constante
acelera	=	asignación
bandera_verde	if	condicional
bandera_amarilla	else if	sino si
bandera_azul	else	sino
vuelta	while	bucle condicional
circuito ... salida ... llegada	for i in a...b	bucle con contador (el límite final se incluye)
bandera_roja	break	romper el bucle
pit_stop	continue	continuar con la siguiente iteración
radio	readLine()	leer entrada del teclado
transmision	print	mostrar en pantalla
boxes	func	función
meta	return	retornar
escuderia	class	clase
ensamble	init	constructor
debut	Clase(...)	crear objeto (nuevo)
auto	self	este objeto
safety_car	do / try	intentar
accidente	catch	capturar
podio	defer / finally	finalmente
penalizacion	throw	lanzar
aliado	&&	y
rival	||	o
contra	!	no
supera	>	mayor que
rezaga	<	menor que
alcanza	>=	mayor o igual
cede	<=	menor o igual
empata	==	igual a
distinto	!=	diferente
gana / pierde	true / false	booleanos
bandera_cuadros	}	fin de bloque
//	//	comentario de línea

3.3 Funciones predefinidas

Además de las palabras reservadas, PitLane incluye cinco funciones predefinidas.

Funciones predefinidas de PitLane

Función	Descripción
longitud(x)	Cantidad de elementos de una lista o de caracteres de un texto
agregar(lista, valor)	Agrega un elemento al final de la lista
a_entero(x)	Convierte a entero; a_entero("abc") es un error de ejecución
a_real(x)	Convierte a real
a_texto(x)	Convierte a texto con las reglas de impresión

3.4 Reglas del lenguaje

Aspecto	Regla
Tipos	Entero, real, texto, booleano (gana / pierde), lista, objeto y función. Una variable declarada sin valor vale nada.
Impresión	Los reales siempre llevan punto (3.0, 2.5). Las listas se muestran como [4, 8, 15], con los textos entre comillas dobles; los objetos como <NombreClase>; una lista cíclica como [<cycle>].
Aritmética	Como en Swift. Entero entre entero es división entera; si algún operando es real, el resultado es real. El operador + concatena cuando algún operando es texto.
Textos	Van entre comillas dobles y admiten las secuencias \n, \t, \" y \\.
Identificadores	Letras (incluidas la ñ y las vocales con tilde), dígitos y _, sin empezar por dígito.
Comentarios	Empiezan con // y llegan hasta el fin de la línea.
Fin de sentencia	Cada sentencia termina en el fin de línea, salvo dentro de ( ) o [ ]: ahí los saltos de línea se ignoran, así que una lista, una llamada o los parámetros de una función pueden repartirse en varias líneas.
Bloques	Se cierran siempre con bandera_cuadros.
Funciones y clases	Se pueden usar antes del punto del archivo donde se definen. Las variables no.
Excepciones	penalizacion lanza cualquier valor. safety_car / accidente / podio captura también los errores de ejecución; el mensaje llega como texto, por ejemplo division by zero.
Entrada	radio() lee una línea de la entrada estándar; si la entrada terminó, devuelve texto vacío.
Archivos fuente	Archivos .txt en UTF-8 (con o sin BOM) o en Windows-1252 (el formato «ANSI» de los editores de Windows).

3.5 Tabla léxica de PitLane

Token	Patrón (regex)	Ejemplo
ID	`[letra_][letra0-9_]*`	contador
INT_NUMBER	`[0-9]+`	20
REAL_NUMBER	`[0-9]+\.[0-9]+`	3.14
STRING	`"([^"\\\n]|\\[nt"\\])*"`	"Mayor que 5"
KW_<PALABRA>	`parrilla|pole|acelera|bandera_verde|...`	bandera_verde
Operadores aritméticos	`[+\-*/%]`	+
Delimitadores	`[()\[\],:.]`	(
Comentario	`//[^\n]*`	// nota (se ignora)
Fin de línea	`\n`	(termina la sentencia)
Espacios	`[ \t\r]+`	(se ignoran)

En el patrón de **ID**, «letra» incluye la ñ y las vocales con tilde. Los operadores relacionales y lógicos se escriben con palabras, por lo que son palabras reservadas: **supera, rezaga, alcanza, cede, empata, distinto, aliado, rival y contra**. Lo mismo ocurre con **gana** y **pierde**.

Reglas del analizador léxico de PitLane:

- Una palabra con forma de identificador es reservada si está en el vocabulario de la Tabla 5; si no, es un `ID`.
- El fin de línea termina la sentencia, salvo dentro de `( )` y `[ ]`, donde se ignora.
- Los comentarios y los espacios se descartan.
- Entre los errores léxicos están el símbolo no reconocido, el texto sin cerrar y la secuencia de escape inválida.

3.6 Programas de ejemplo

Los tres programas de ejemplo del enunciado, escritos en PitLane. El primero usa condicionales:

```
parrilla x acelera 20
bandera_verde x supera 5:
    transmision("Mayor que 5")
bandera_azul:
    transmision("Menor o igual a 5")
bandera_cuadros
```

Salida:

```
Mayor que 5
```

El segundo usa una función, una lista y un bucle con contador:

```
// Suma de los elementos de una lista
boxes sumar(lista, n):
    parrilla total acelera 0
    circuito i salida 0 llegada n - 1:
        total acelera total + lista[i]
    bandera_cuadros
    meta total
bandera_cuadros
parrilla numeros acelera [4, 8, 15, 16]
parrilla resultado acelera sumar(numeros, 4)
bandera_verde resultado supera 40 aliado resultado rezaga 50:
    transmision("The sum is " + resultado)
bandera_azul:
    transmision("Out of range")
bandera_cuadros
```

Salida:

```
The sum is 43
```

El tercero usa una clase, un constructor, una excepción lanzada por el programa y el bloque de manejo de errores:

```
escuderia Cuenta:
    ensamble(titular, saldo):
        auto.titular acelera titular
        auto.saldo acelera saldo
    bandera_cuadros
    boxes retirar(valor):
        bandera_verde valor supera auto.saldo:
            penalizacion "Insufficient balance"
        bandera_cuadros
        auto.saldo acelera auto.saldo - valor
        meta auto.saldo
    bandera_cuadros
bandera_cuadros
parrilla c acelera debut Cuenta("Ana", 100)
safety_car:
    transmision("New balance: " + c.retirar(30))
    transmision("New balance: " + c.retirar(200))
accidente error:
    transmision("Error: " + error)
podio:
    transmision("Operation finished")
bandera_cuadros
```

Salida:

```
New balance: 70
Error: Insufficient balance
Operation finished
```

Un programa con recursión, que usa un bucle con contador:

```
boxes factorial(n):
    bandera_verde n cede 1:
        meta 1
    bandera_cuadros
    meta n * factorial(n - 1)
bandera_cuadros
circuito i salida 1 llegada 5:
    transmision("factorial de " + i + " = " + factorial(i))
bandera_cuadros
```

Salida:

```
factorial de 1 = 1
factorial de 2 = 2
factorial de 3 = 6
factorial de 4 = 24
factorial de 5 = 120
```

4. Gramática de PitLane (EBNF)

4.1 Convenciones

La gramática se escribe en EBNF (ISO/IEC, 1996). `{ }` repite cero o más veces, `[ ]` es opcional, `|` es una alternativa, `( )` agrupa, `"..."` es un terminal y `NL` es un salto de línea. Dentro de `( )` y `[ ]` los saltos de línea se ignoran, por lo que no aparecen en las reglas.

4.2 Reglas

Estructura del programa, declaraciones y asignación:

```
(* Estructura y sentencias *)
programa     = { sentencia } ;
sentencia    = ( declaracion | asignacion | si | vuelta | circuito
               | romper | continuar | retorno | transmision
               | funcion | clase | intento | lanzar | llamada ) NL ;
bloque       = { sentencia } ;
inicio       = ":" NL ;
cierre       = "bandera_cuadros" ;

(* Declaraciones y asignacion *)
declaracion  = ( "parrilla" | "pole" ) ID [ "acelera" expr ] ;
asignacion   = destino "acelera" expr ;
destino      = ID
             | ID "[" expr "]"
             | "auto" "." ID
             | postfijo "." ID ;
llamada      = expr ;
```

Control de flujo, entrada y salida:

```
(* Control de flujo *)
si           = "bandera_verde" expr inicio bloque
               { "bandera_amarilla" expr inicio bloque }
               [ "bandera_azul" inicio bloque ]
               cierre ;
vuelta       = "vuelta" expr inicio bloque cierre ;
circuito     = "circuito" ID "salida" expr "llegada" expr inicio bloque cierre ;
romper       = "bandera_roja" ;
continuar    = "pit_stop" ;

(* Entrada y salida *)
transmision  = "transmision" "(" expr ")" ;
leer         = "radio" "(" ")" ;
```

Funciones, clases y excepciones:

```
(* Funciones y clases *)
funcion      = "boxes" ID "(" [ params ] ")" inicio bloque cierre ;
params       = ID { "," ID } ;
retorno      = "meta" [ expr ] ;
clase        = "escuderia" ID inicio { miembro } cierre ;
miembro      = ( constructor | funcion ) NL ;
constructor  = "ensamble" "(" [ params ] ")" inicio bloque cierre ;

(* Excepciones *)
intento      = "safety_car" inicio bloque
               "accidente" ID inicio bloque
               [ "podio" inicio bloque ]
               cierre ;
lanzar       = "penalizacion" expr ;
```

Expresiones, de menor a mayor precedencia, y terminales léxicos:

```
(* Expresiones, de menor a mayor precedencia *)
expr         = o_logico ;
o_logico     = y_logico { "rival" y_logico } ;
y_logico     = igualdad { "aliado" igualdad } ;
igualdad     = relacional { ( "empata" | "distinto" ) relacional } ;
relacional   = suma { ( "supera" | "rezaga" | "alcanza" | "cede" ) suma } ;
suma         = termino { ( "+" | "-" ) termino } ;
termino      = unario { ( "*" | "/" | "%" ) unario } ;
unario       = ( "contra" | "-" ) unario | postfijo ;
postfijo     = primario { "(" [ args ] ")" | "[" expr "]" | "." ID } ;
args         = expr { "," expr } ;
primario     = INT | REAL | STRING | "gana" | "pierde"
             | ID | "auto"
             | "(" expr ")"
             | "[" [ args ] "]"
             | "debut" ID "(" [ args ] ")"
             | leer ;

(* Terminales lexicos *)
ID     = letra { letra | digito | "_" } ;
INT    = digito { digito } ;
REAL   = digito { digito } "." digito { digito } ;
STRING = '"' { caracter_comun | escape } '"' ;
escape = "\" ( "n" | "t" | '"' | "\" ) ;
(* Comentario: "//" hasta fin de linea, se ignora *)
```

4.3 Precedencia y asociatividad

Cada nivel de la gramática de expresiones corresponde a una fila de la Tabla 9, de menor a mayor precedencia. Los operadores binarios se asocian a la izquierda porque las reglas usan repetición `{ }`.

Precedencia de los operadores

Nivel	Operadores	Tipo
1	rival	binario, asocia a la izquierda
2	aliado	binario, asocia a la izquierda
3	empata, distinto	binario, asocia a la izquierda
4	supera, rezaga, alcanza, cede	binario, asocia a la izquierda
5	+, -	binario, asocia a la izquierda
6	*, /, %	binario, asocia a la izquierda
7	contra, - (negativo)	prefijo
8	( ) (llamada), [ ] (índice), . (atributo)	posfijo

4.4 Restricciones de contexto

Hay reglas que la gramática libre de contexto no puede expresar. El análisis sintáctico las verifica antes de ejecutar cualquier instrucción y las informa como errores de sintaxis con su línea:

- bandera_roja y pit_stop solo pueden aparecer dentro de un bucle ('bandera_roja' outside loop). Un bucle exterior no los habilita dentro de una función definida en él.
- meta solo puede aparecer dentro de una función o de un ensamble ('meta' outside function).
- Los parámetros de una función no pueden repetirse (duplicate parameter 'a').
- Una escuderia no puede tener dos métodos con el mismo nombre (duplicate method 'm') ni un segundo ensamble.

4.5 Notas para el analizador sintáctico

- **Descenso recursivo:** cada regla de la gramática se convierte en una función del analizador.
- **Asignación o llamada:** para distinguir x acelera 5 de una llamada suelta, se analiza primero un postfijo y luego se mira si sigue acelera. Cualquier expresión es válida como sentencia (regla llamada).
- **Concatenación:** + suma números y une textos; el intérprete decide según el tipo de los operandos.
- **Llamadas en expresiones:** postfijo permite escribir c.retirar(30) dentro de transmision("..." + c.retirar(30)).
- **Errores de ejecución:** la división por cero y el índice fuera de rango se lanzan como errores que accidente puede capturar.

5. Errores y límites del procesador

5.1 Formato de los errores

Los errores léxicos, sintácticos y de ejecución se imprimen por la salida estándar, indican la línea y terminan el proceso con código de salida 1. Lo que el programa ya había impreso se conserva. Ejemplos:

```
Lexical error on line 2: unrecognized symbol '#'
Lexical error on line 3: unterminated string
Lexical error on line 2: invalid escape sequence '\q'
Syntax error on line 4: expected 'bandera_cuadros'
Syntax error on line 3: expected expression
Syntax error on line 5: 'bandera_roja' outside loop
Syntax error on line 2: duplicate parameter 'a'
Runtime error on line 4: division by zero
```

Otros errores de ejecución son `index out of range`, `undeclared variable 'x'`, `cannot assign to constant 'x'`, `'x' already declared`, `expected N arguments but got M`, `not callable`, `stack overflow`, `undefined attribute 'x'`, `undefined method 'm'`, `undefined class 'X'` y `condition must be boolean`. Una `penalizacion` que nadie captura se informa como `Runtime error on line N:` seguido del valor lanzado.

5.2 Límites de ejecución

Para evitar bloqueos y consumo excesivo de recursos, el procesador impone los límites de la Tabla 10. El tiempo se mide como tiempo de CPU del intérprete, así que la espera de **radio()** no cuenta.

Límites del procesador

Límite	Valor	Al superarlo
Tamaño del archivo fuente	1 000 000 de bytes	Código de salida 1 y mensaje por la salida de errores
Pasos de ejecución	100 000 000	execution limit exceeded, con la línea en curso
Tiempo de CPU	4 s (respaldo: 5 s)	execution limit exceeded; el respaldo informa time limit exceeded
Anidamiento	128 niveles (en la ejecución, dentro de cada función)	Error con la línea en curso
Recursión	1000 llamadas anidadas	stack overflow
Memoria	256 MiB de pico residente	memory limit exceeded
Línea de radio(), texto construido y salida	1 000 000 de bytes cada uno	Error con la línea en curso
Comparación de valores	1 000 000 de pares	Error con la línea en curso

6. Alcance y estado

El procesador cubre los requisitos mínimos del enunciado: tipos de datos, variables, comentarios, expresiones aritméticas, relacionales y lógicas, concatenación, condicionales, bucles, funciones, listas, clases, objetos y excepciones. La carpeta **ejemplos**/ contiene una prueba por requisito y archivos con errores léxicos, sintácticos y de ejecución; el script **probar.sh** los ejecuta y compara cada salida con la esperada.

De los puntos opcionales, está implementada la **recursión**. No están implementados la interfaz gráfica, la integración con bases de datos, la herencia, el do-while, el switch ni la detección de errores semánticos en una fase previa a la ejecución: los errores de tipo, de variables no declaradas y de constantes se detectan al ejecutar.

7. Referencias

Apple Inc. (s. f.). *The Swift Programming Language*. Swift.org. https://docs.swift.org/swift-book/

International Organization for Standardization & International Electrotechnical Commission. (1996). *Information technology — Syntactic metalanguage — Extended BNF* (ISO/IEC 14977:1996).

Morales, J. H. (s. f.). *Language Processors: Design and Implementation of a Mini Programming Language* [Enunciado del proyecto]. Universidad Autónoma de Manizales.